Lecture 5

Cache Memory

Von Neumann bottleneck

- The separation of memory and CPU is often called the von Neumann bottleneck, since the interconnect determines the rate at which instructions and data can be accessed.
- The potentially vast quantity of data and instructions needed to run a program is effectively isolated from the CPU.
- In 2021, CPUs are capable of executing instructions more than one hundred times faster than they can fetch items from main memory.
- To address the von Neumann bottleneck and, more generally, improve computer performance, many modifications to the basic von Neumann architecture has been done.

Modifications to the von Neumann model

- As we noted earlier, since the first electronic digital computers were developed back in the 1940s, computer scientists and computer engineers have made many improvements to the basic von Neumann architecture.
- Many are targeted at reducing the problem of the von Neumann bottleneck, but many are also targeted at simply making CPUs faster.
- In this section, we'll look one of these improvements: caching.

Principle of Locality

- Two different types of locality have been observed:

a) *Temporal locality* states that recently accessed items are likely to be accessed in the near future.

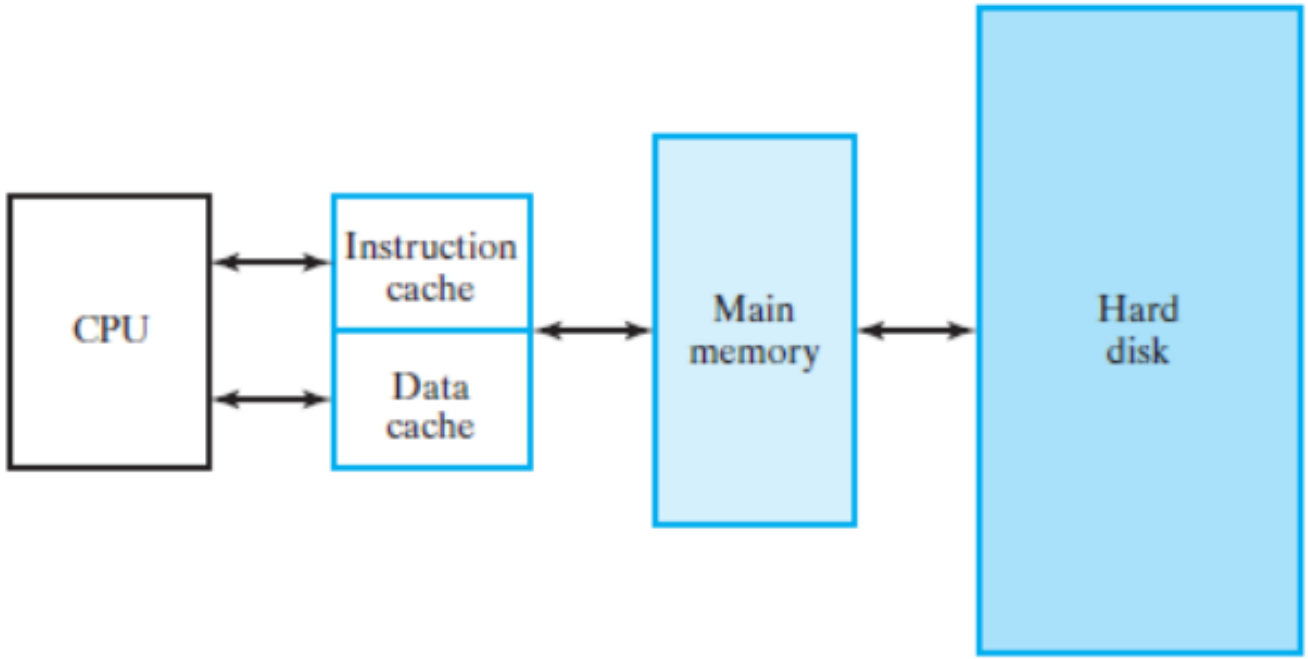
b) Spatial locality says that items whose addresses are near one another tend to be referenced close together in time.

MEMORY HIERARCHY

- The lowest level of the hierarchy is a small, fast memory called a *cache*.
- At the next level upward in the hierarchy is the *main memory*.
- The main memory serves directly most of the CPU instructions and operand fetches not satisfied by the cache.
- At the top level of the hierarchy is the *hard drive*, which is accessed only in the very infrequent cases where a CPU instruction or a operand fetch is not found in main memory.

Example of Memory Hierarchy with Two Caches

- The use of these two caches permits one instruction and one operand to be fetched, or one instruction to be fetched and one result to be stored, in a single clock cycle if the caches are fast enough.



Multiple-Level Caches

- Two levels of cache, often referred to as L1 and L2, with L1 closest to the CPU, are often used.
- In order to satisfy the demand of the CPU for instruction and operands, a very fast L1 cache is needed.
- The L1 cache is placed in the processor IC together with the CPU and is referred to as the *internal cache.*
- The L1 cache can be designed to specific CPU access needs including the possibility of separate instruction and data caches.
- A larger L2 cache is added outside the processor IC. If more space is available in the IC, then the L2 cache can also be an internal cache.
- L2 rather than providing instructions and operands to a CPU, it primarily provides instructions and operands to the first-level cache L1. The L2 cache is accessed only on L1 misses,

Cache Memory

- To illustrate the concept of cache memory, we assume a very small cache of:
- Eight (32-bit) words,
- A small main memory with 1 KB (256 words of 32-bit).
- The cache address lines = 3 bits,
- The memory address lines = 10 bits,
- Out of the 256 words of size 32 bit / word, in the main memory, only 8 at a time may lie in the cache.
- In order for the CPU to address a word in the cache, there must be information in the cache to identify the address of the word in main memory

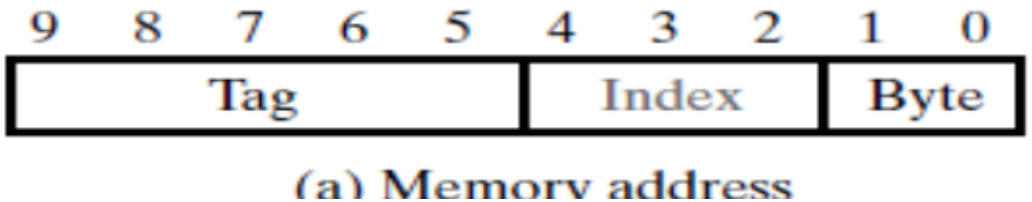
Cache Mappings

- Mapping is between the main memory address and the cache address.
- There are many cache mapping technologies:
  - Direct cache Mapping
  - Fully Associative Cache mapping
  -Associative Memory
  - Set-associative Cache mapping

1- Direct Cache Mapping

- The main memory address is partitioned into three sections:
  - 1- No., of Bytes
  - 2- Index
  - 3 Tag

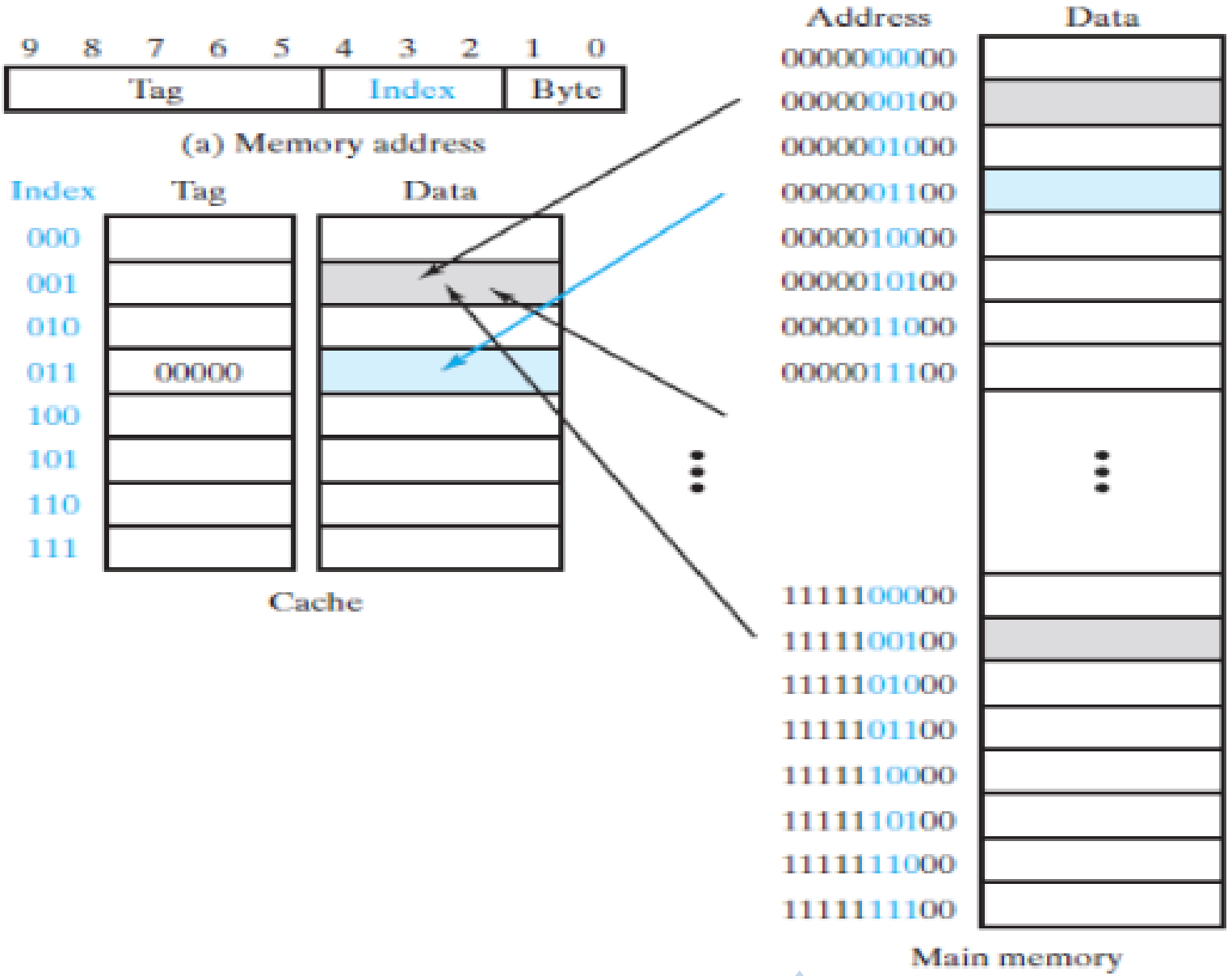
For Example:



(a) Memory address

Cont.,

- Byte Field: has the lowest two bits to be able to map 4 succeeded bytes from the main memory to one cache memory location (32-bit size).
- Index Field: has the next three bits because the cache memory has eight memory locations
- Upper 5 bits of the main memory address, called the *tag*, are stored in the cache along with the data.


9 8 7 6 5 4 3 2 1 0
Tag
Index
Byte
(a) Memory address
Index
Tag
Data
000
001
010
011
00000
100
101
110
111
Cache
Address
Data
0000000000
0000000100
0000001000
0000001100
0000010000
0000010100
0000011000
0000011100
1111100000
1111100100
1111101000
1111101100
1111110000
1111110100
1111111000
1111111100
Main memory

Mapping Process

- Suppose that the CPU is to fetch an instruction from location 000001100 in main memory.
- The CPU start fetches in his cache, as follows:

  - **Compare Index ? sure matched**

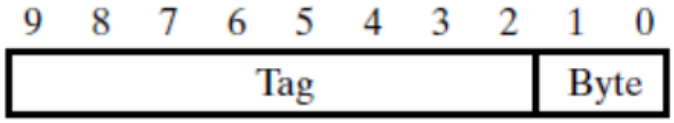
  - **Compare Tag**, If tag matched, called cache hit, else called cache miss and looks for the address in the main memory

Cont.,

- In direct mapping, by dividing 256 words in the main memory by 8 words in cache memory, we will find that for every 2^5 addresses in main memory, a single address can map to the single address in the cache that matches the same index at a time.
- Only one of the 2^5 addresses can have its word in cache address at any time.(disadvantage)

2- *fully associative* mapping

- In contrast to direct mapping, suppose that we let locations in main memory map into an arbitrary location in the cache.
- Then any location in memory can be mapped to any one of the eight addresses in the cache.
- This means that the tag will now be the full main memory word address.



(a) Memory address

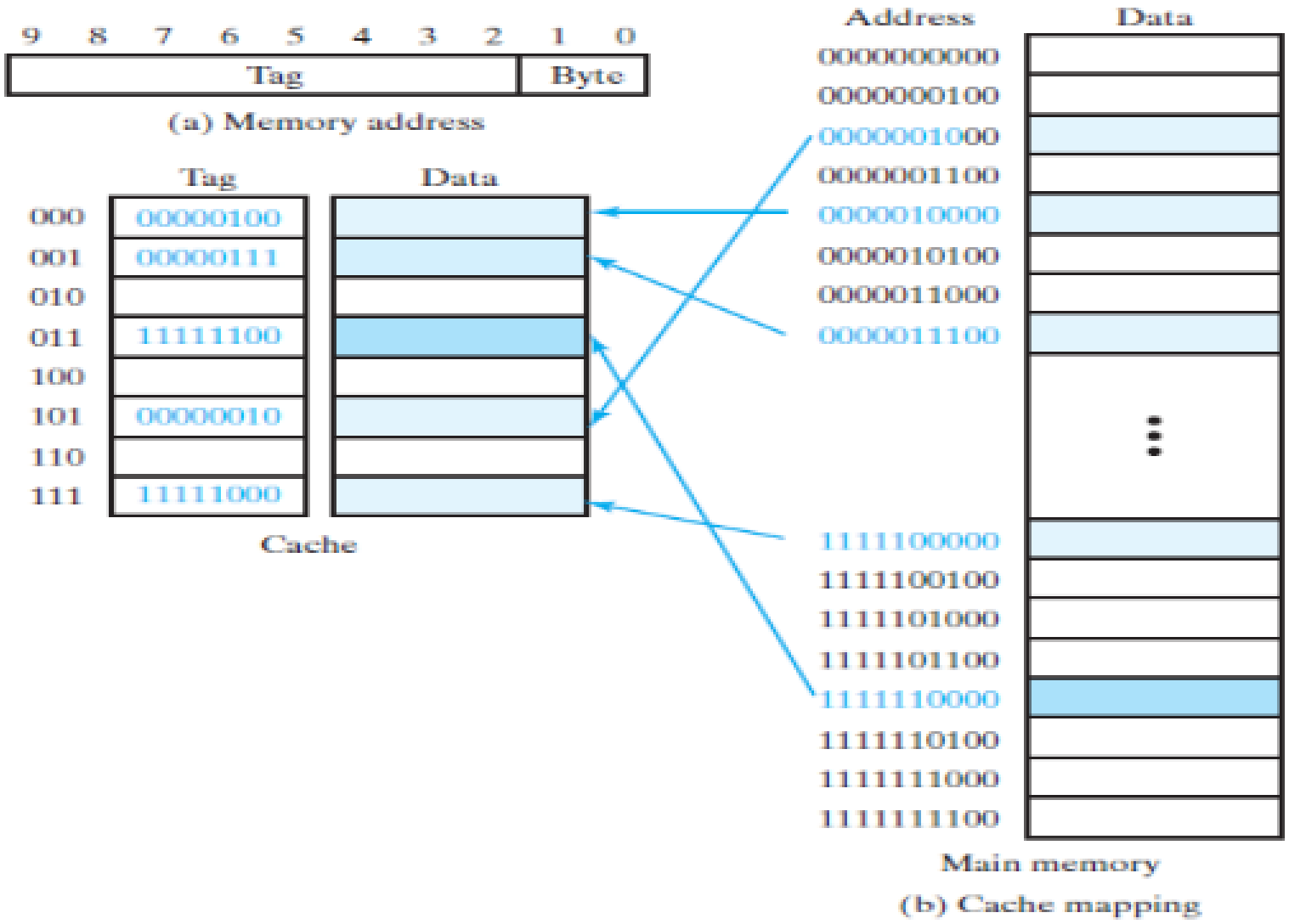


FIGURE 12-4
Fully Associative Cache

Mapping Process

- Suppose that the CPU is to fetch an instruction from location 0000010000 in cache memory:
- The cache must compare 00000100 to each of its eight tags.

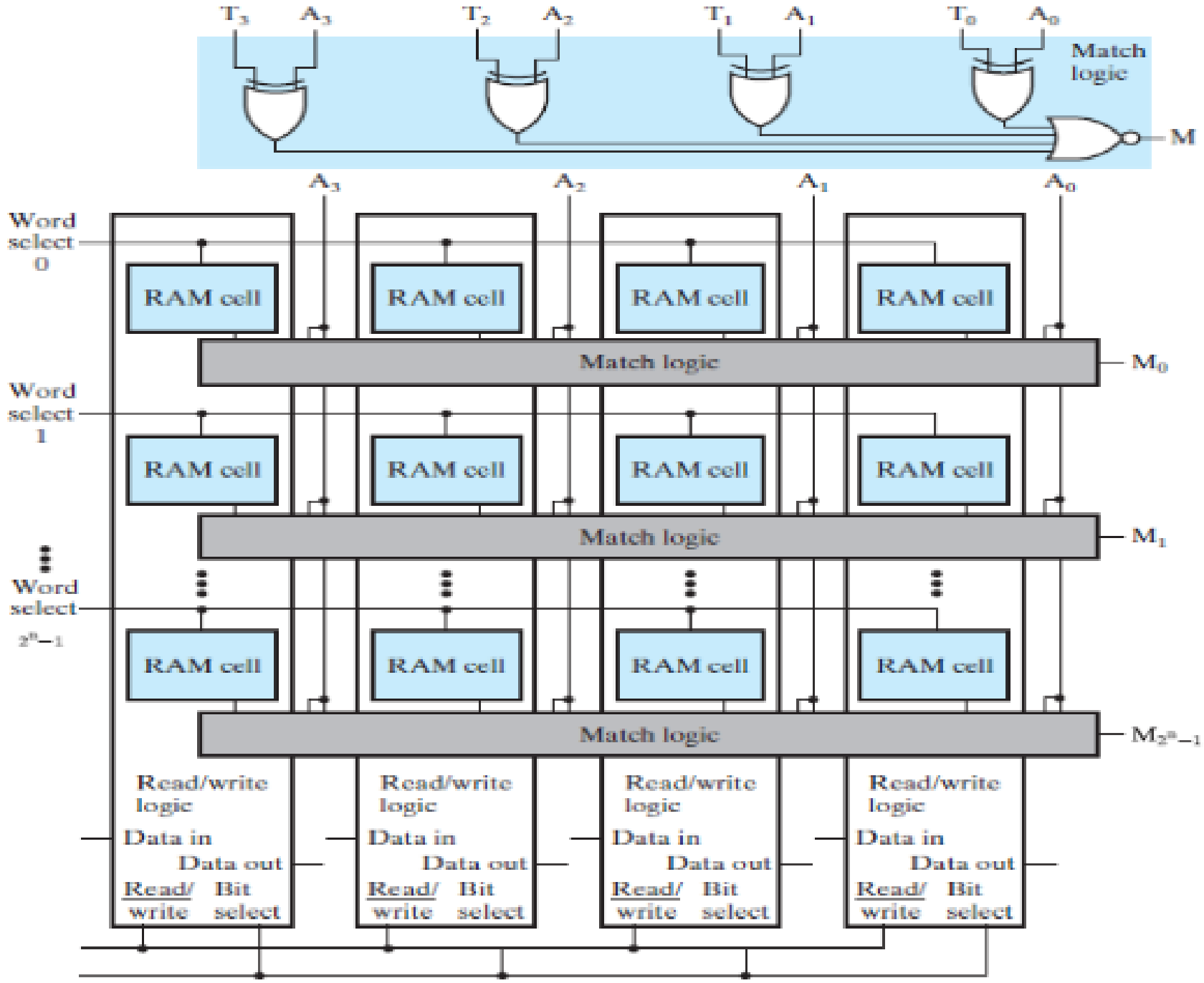
- One way to do this is to successively read each tag and the associated word from the cache memory and compare the tag to 00000100.
- If a match occurs, a cache hit occurs, and the cache control then places the word on the bus to the CPU, completing the fetch operation.
- If the tag fetched from the cache is not matched, then there is a tag mismatch, and the cache control fetches the next successive tag and word.

Cont.,

- In the worst case, a match on the tag in cache address, eight fetches from the cache are required before the cache hit occurs.
- At 2 ns a fetch, this requires at least 16 ns, about half the time it would take to obtain the instruction from main memory (cons).
- So successive reads of tags and words from the cache memory to find a match is not a very desirable approach

3- Associative Memory

- A structure called associative memory implements the tag portion of the cache memory.
- An associative memory is known as content-addressable memory (CAM). The subfield chosen to address the memory (TAG) is called the key.
- The idea is to minimize the fetching time by adding an associative memory for fetching Tags.
- Let : T □ is the Tag address, and A □ is the applied CPU address.
- The matching logic circuit does an equality comparison between the Tag ‘T’, and the applied address ‘A’, for the eight cache memory locations at the same time.


T_3 A_3
T_2 A_2
T_1 A_1
T_0 A_0
Match logic
M
A_3
A_2
A_1
A_0
Word select 0
Word select 1
Word select 2^n-1
RAM cell
Match logic
M_0
M_1
M_{2^n-1}
Read/write logic
Data in
Data out
Read/ write
Bit select

Cont.,

- The match logic does an equality comparison or match between the tag *T* and the applied address *A* from the CPU.
- The match logic for each tag is composed of an exclusive-OR gate for each bit and a NOR gate that combines the outputs of the exclusive-ORs.
- If all of the bits of the tag and the address match, then the outputs of all the exclusive-ORs are 0 and the NOR output is a 1, indicating a match.
- If there is a mismatch between any of the bits in the tag and the address, then at least one exclusive-OR has a 1 output, which causes the output of the NOR gate to be 0, indicating a mismatch.
- The drawback of the method is the matching logic circuit cost and size.

4- Set-associative Cache mapping

- An alternative mapping that has better performance and eliminates the cost of most of the matching logic is a compromise between a direct-mapped cache and a fully associative cache.
- For such a mapping, lower-order address bits (Index) act much as they do in direct mapping; however, for each combination of lower-order address bits, instead of having one location, there is a *set* of ***s*** locations.

Two-Way set Associative Cache

- For example, if the *set size* equals two, then two tags and the two accompanying data words are read simultaneously.
- The tags are then simultaneously compared to the CPU-supplied address using just two matching logic structures.
- Eight cache locations are arranged in four rows of two locations each.
- The rows are addressed by a 2-bit index and contain tags made up of the remaining six bits of the main memory address.

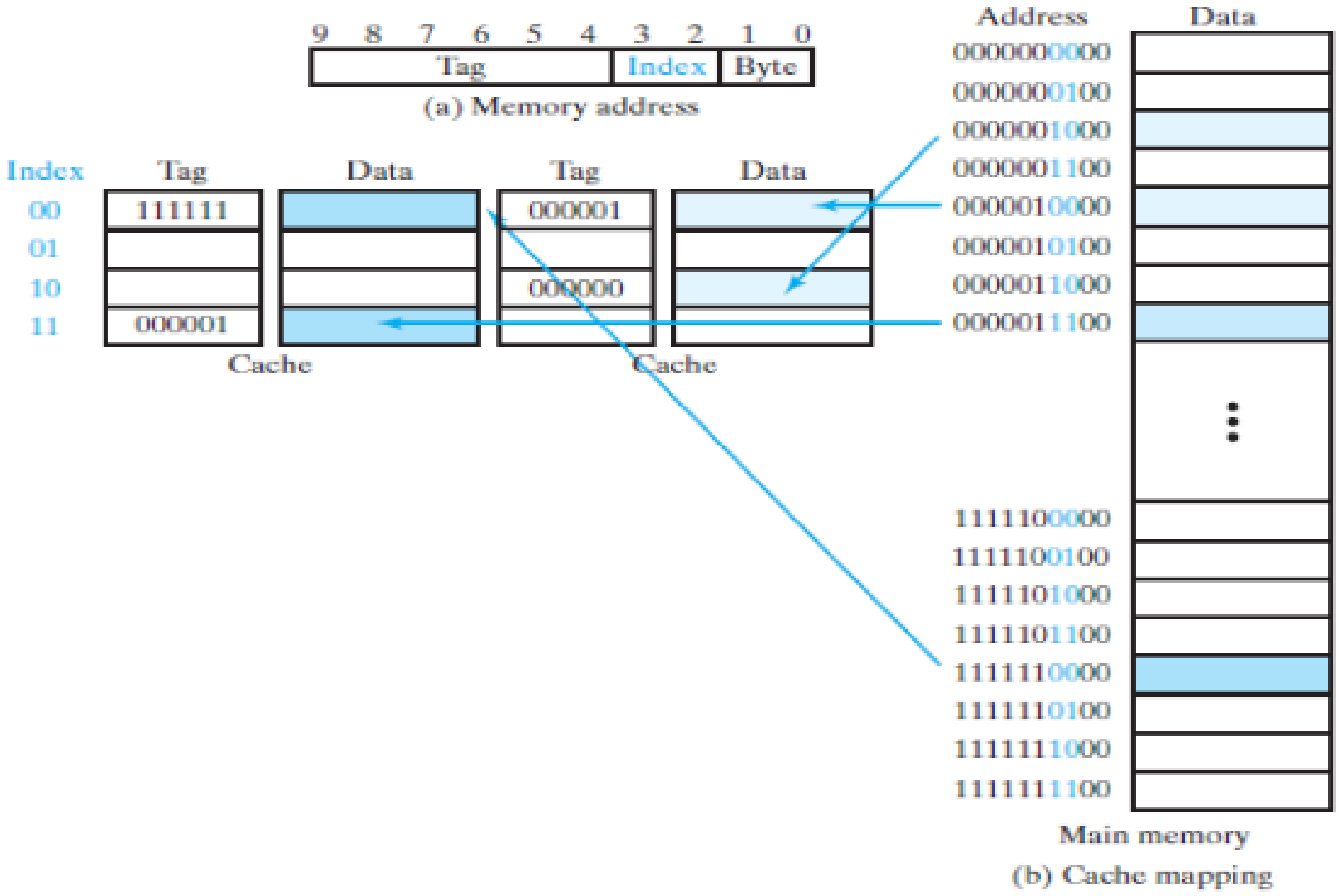
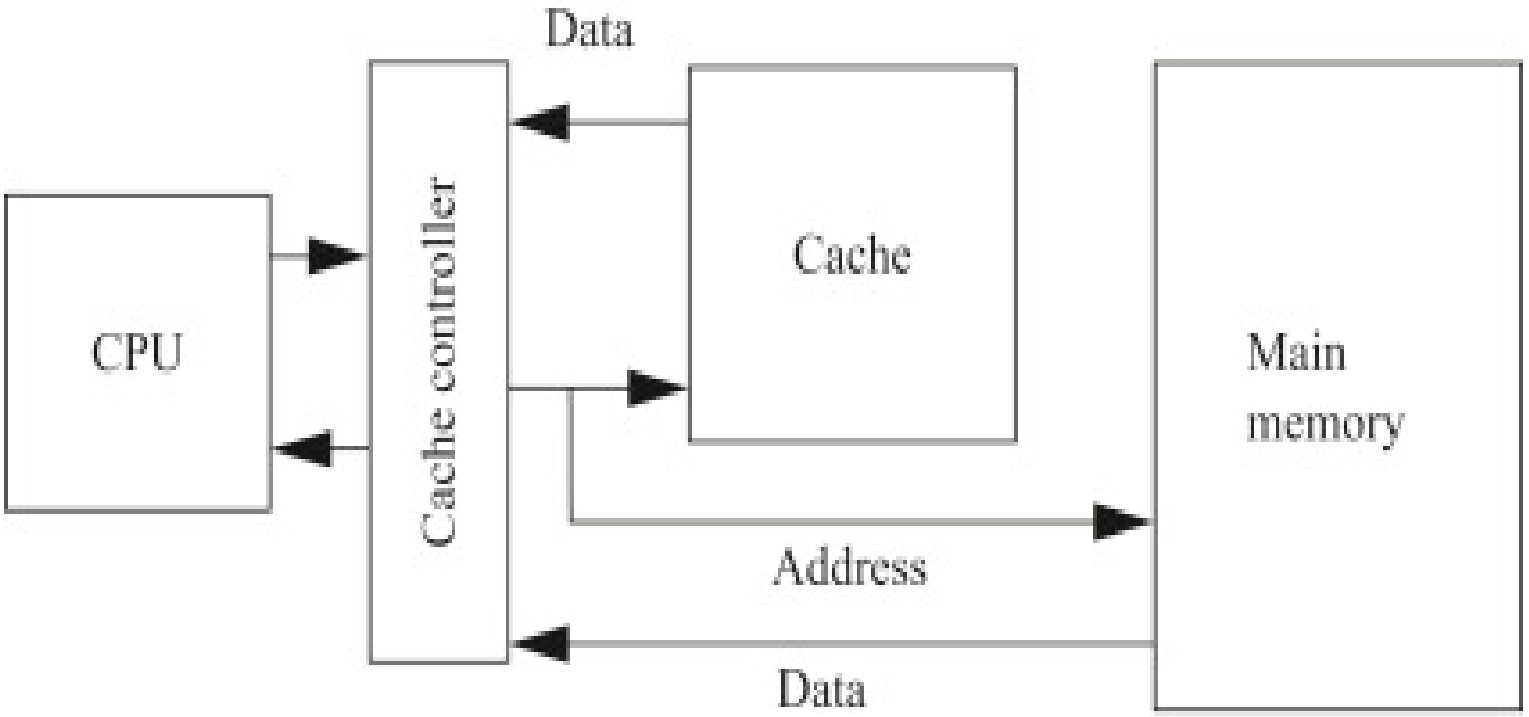


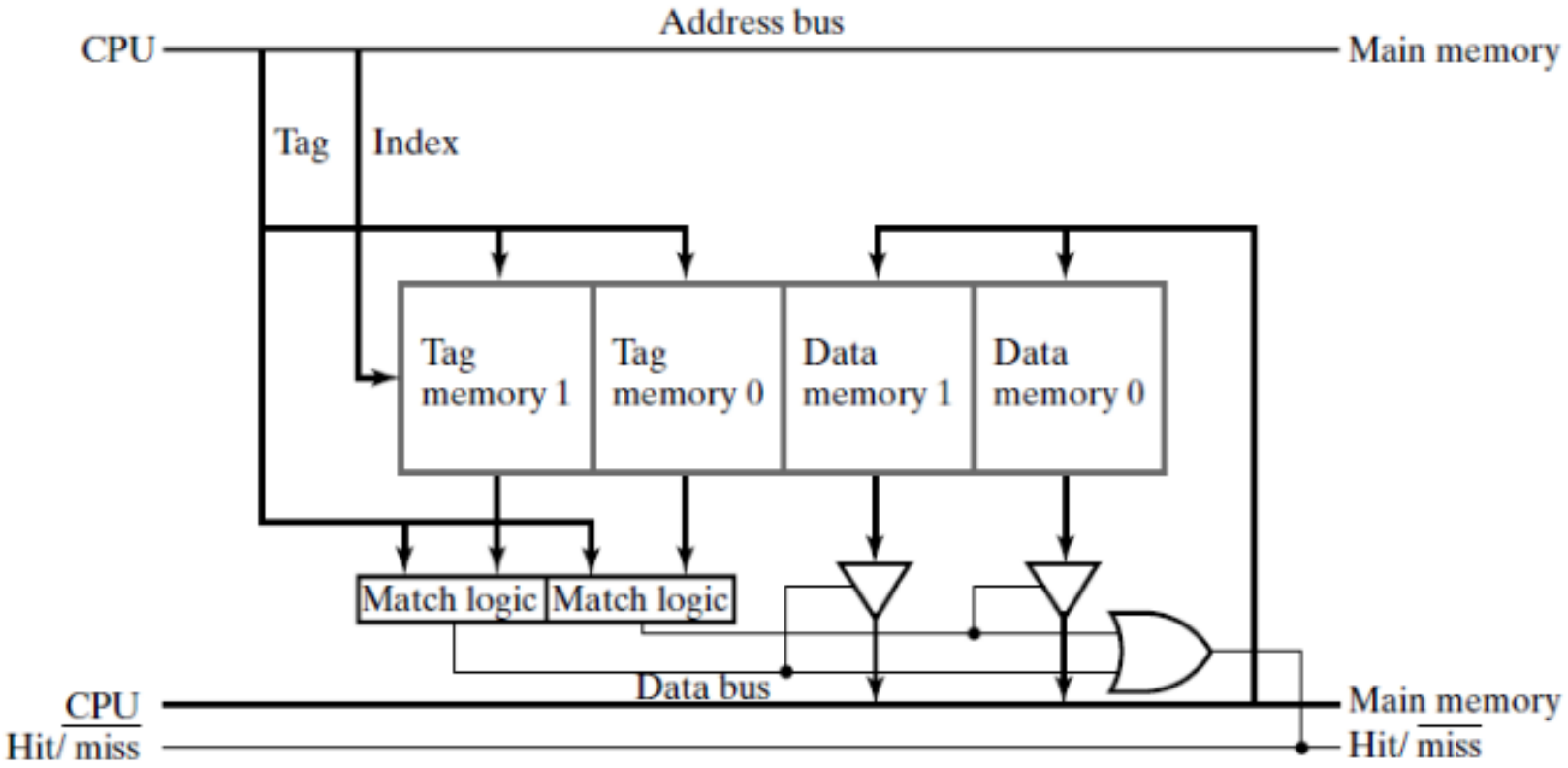
FIGURE 12-6
Two-Way Set-Associative Cache

Mapping Process

- The index is used to address each row of the cache memory.
- The two tags read from the tag memories are compared to the tag part of the address on the address bus from the CPU.
- If a match occurs, then the three-state buffer on the corresponding data memory output is activated, placing the data onto the data bus to the CPU as shown in the following figure.
- In addition, the match signal causes the output of the Hit/miss OR gate to become 1, indicating a hit.
- If a match does not occur, then Hit/miss is 0, informing the main memory that it must supply the word to the CPU, and informing the CPU that the word will be delayed.

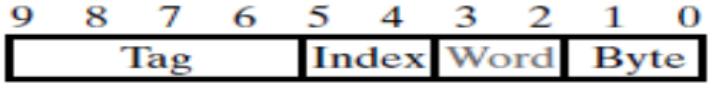
Cache Controller



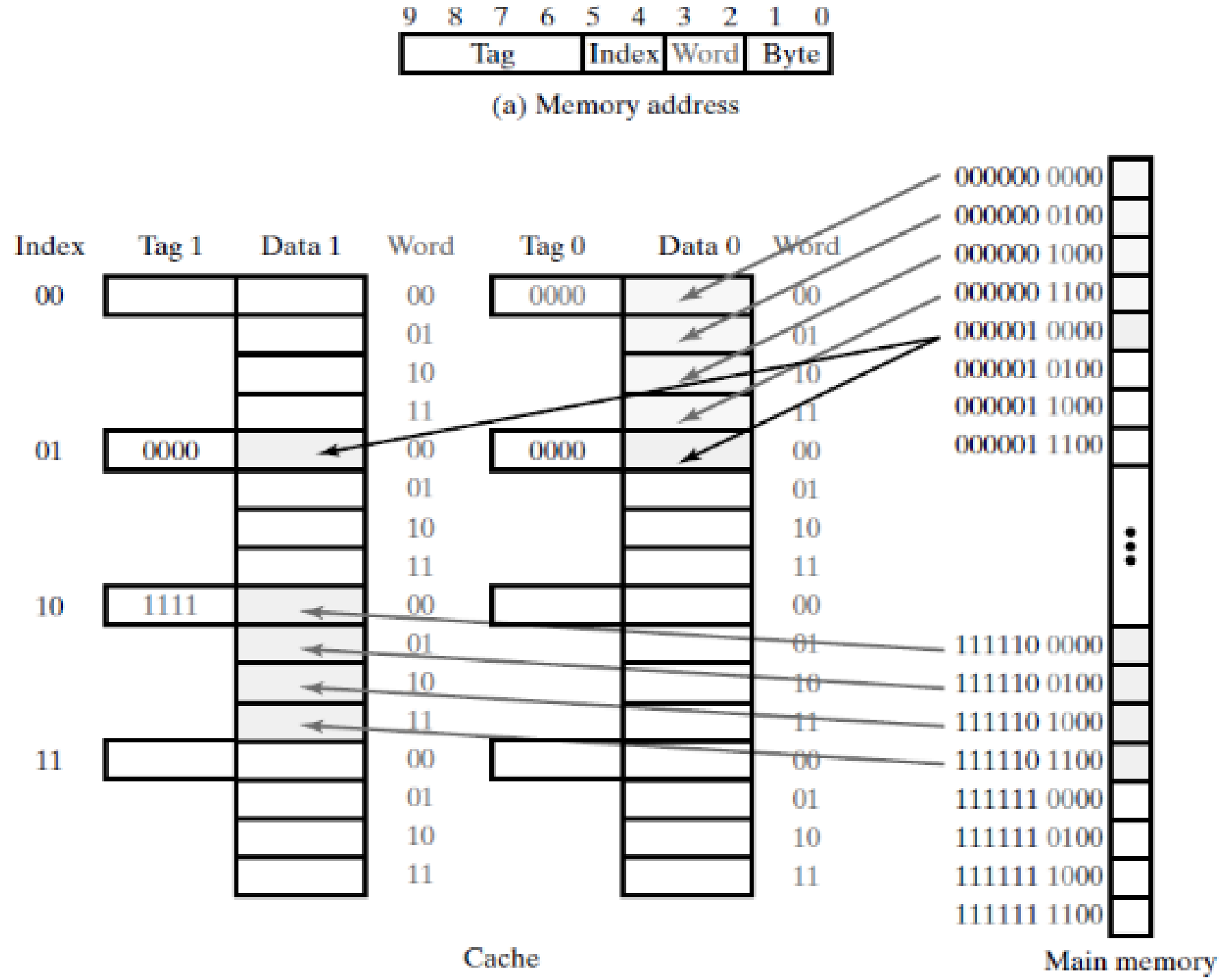

Address bus
CPU
Main memory
Tag
Index
Tag memory 1
Tag memory 0
Data memory 1
Data memory 0
Match logic
Match logic
Data bus
CPU
Hit/ miss
Main memory
Hit/ miss

Line Size

- In real caches, spatial locality is to be exploited, so additional words close to the one addressed are included in the cache entry.
- Then, rather than a single word being fetched from main memory when a cache miss occurs, a block of *l* words called a *line* is fetched.
- Bits 2 and 3, the Word field, are used to address the word within the line. In this case, two bits are used, so there are four words per line.



(a) Memory address


9 8 7 6 5 4 3 2 1 0
Tag
Index
Word
Byte
(a) Memory address
Index
Tag 1
Data 1
Word
Tag 0
Data 0
Word
00
01
10
11
0000
1111
000000 0000
000000 0100
000000 1000
000000 1100
000001 0000
000001 0100
000001 1000
000001 1100
111110 0000
111110 0100
111110 1000
111110 1100
111111 0000
111111 0100
111111 1000
111111 1100
Cache
Main memory

Cache Replacement Approaches

- In addition to selecting a cache mapping, the cache designer must select a replacement approach that determines the location in the cache to be used for the incoming tag and data.
- One possibility is to select a *random replacement* location
- A somewhat more thoughtful approach is to use a first-in, first-out (FIFO) location.
- An approach that appears to attack the replacement problem even more directly is the *least recently used* (LRU) location approach.

Cache Write Methods

- We have focused so far on reading instructions and operands from the cache. What happens when a write occurs?
- Following are three possible write actions from which we can select:

  1. Write the result into main memory.

  2. Write the result into the cache.

  3. Write the result into both main memory and the cache.

Cont.,

- Various realistic cache write methods employ one or more of these actions.
- Such methods fall into two main categories:
  - Write-through.
  - Write-back.
- In cache memory, "write back" and "write through" are two different strategies for managing how data is written to both the cache and the underlying main memory.

Write Through

- **Definition**: In the write-through cache strategy, every time data is written to the cache, it is also written simultaneously to the main memory.
- **Advantages**:
  - Simplicity: Easy to implement and understand.
  - Data Consistency: The main memory always contains the most up-to-date data.
- **Disadvantages**:
  - Performance: Slower write operations because every write operation involves accessing both cache and main memory.
  - Increased memory traffic: More frequent writes to the main memory can lead to higher bandwidth usage.

Write Back

- **Definition**: In the write-back cache strategy, data is written to the cache first, and updates are only written to main memory later, typically when the cache line is evicted.
- **Advantages**:
  - Performance: Faster write operations since the write to main memory is delayed.
  - Reduced memory traffic: Fewer writes to main memory can improve overall system performance, especially for applications with frequent writes.
- **Disadvantages**:
  - Complexity: More complex to manage, as the system must track which cache lines have been modified.
  - Potential for Data Loss: If the system crashes before the cache is written back to memory, changes could be lost.

Assignment 1

- Explain the concept of Cache Coherence.

Assignment 2

- A CPU produces the following sequence of read addresses in hexadecimal: 54, 58, 104, 5C, 108, 60, F0, 64, 54, 58, 10C, 5C, 110, 60, F0, 64. Supposing that the cache is empty to begin with, and assuming an LRU replacement, determine whether each address produces a hit or a miss for each of the following caches: (a) direct mapped (b) fully associative , and (c) two-way set associative.

Assignment #3

- A two-way set-associative cache in a system with 32-bit addresses has four 4-byte words per line and a capacity of 1 MB. Addressing is to the byte level. How many bits are there in the index and the tag?

Sheet

- Text Book ch., 12